KOÇ ÜNİVERSİTESİ
COMPUTER ENGINEERING

COMP132: Advanced Programming

Programming Project Report

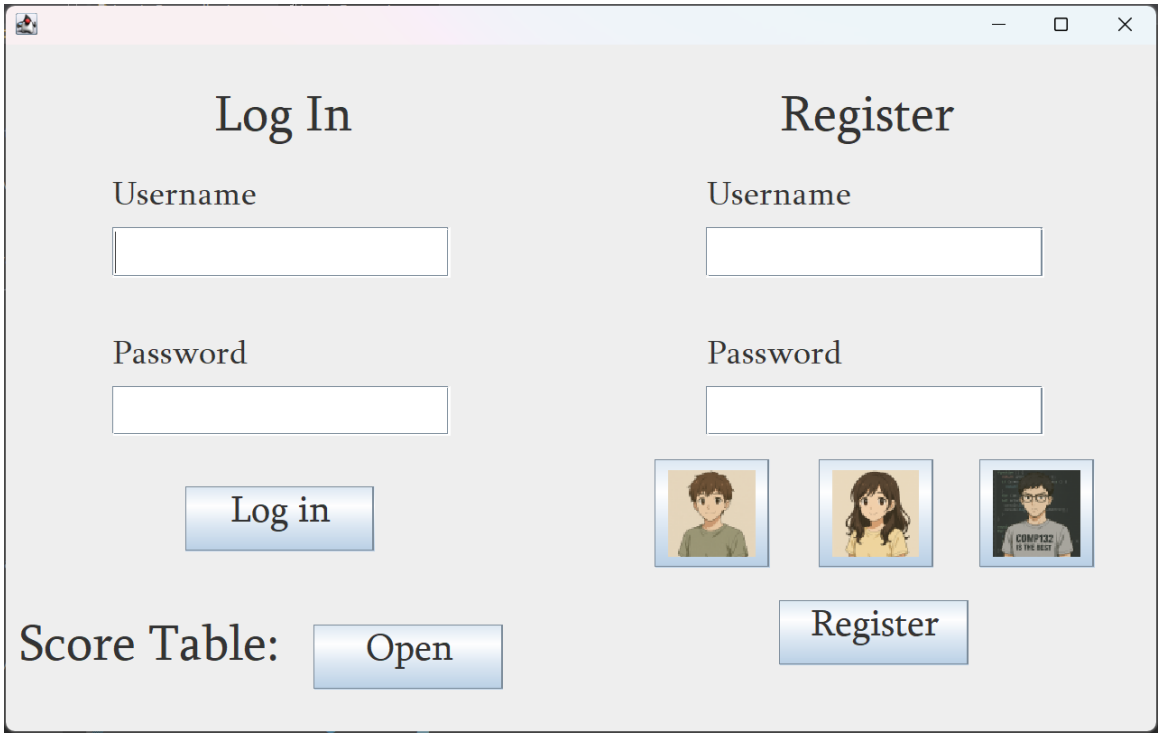
Knowledge Siege: Simulation Design and Development

Elnur Ayyubov, 88909

Spring 2025

General Demo Information

By running the application, a simple login page opens up:



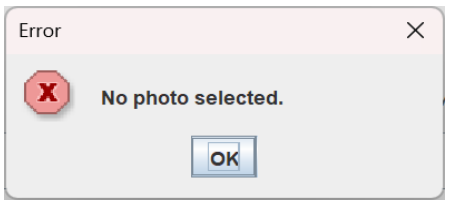
Login and Register Frame

In this page, user can either log in to an existing account, register a new account, or view the score table of the users. A pre-registered user username is *elnur* with password *123*, *omar* – *om@r4k*, *maqa* – *prostomaqa1!*. To log in to that account, type the username to username field and password to password field in login part of the screen. If either of the fields are empty or given username with password is not in database, it will show according error message.

Username or password field is empty

Username or password is wrong

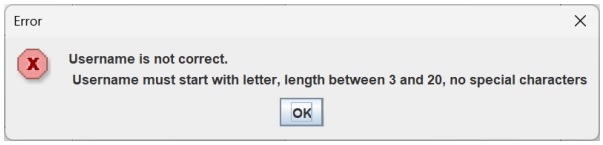
To register a user must type a new username and password to fields accordingly and select one of the provided images. If the fields are empty, none of the images are selected, or username is already taken by other user, the program will show according error messages.



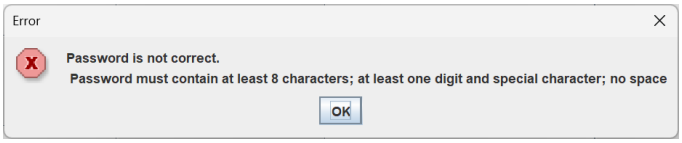
User hasn't selected photo before registering

Trying to register user with already taken username

Also, there is regular expression is used to validate username and password before registering. Username must start with letter, has length from 3 to 20, has no special characters. Password must have length at least 8 characters, contain at least one digit and one special character, no space. Here is the error messages that indicate it:

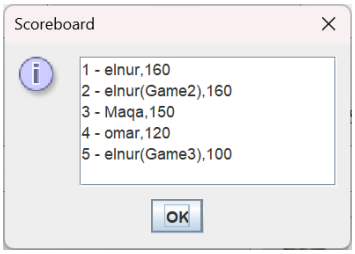


Username does not match regex



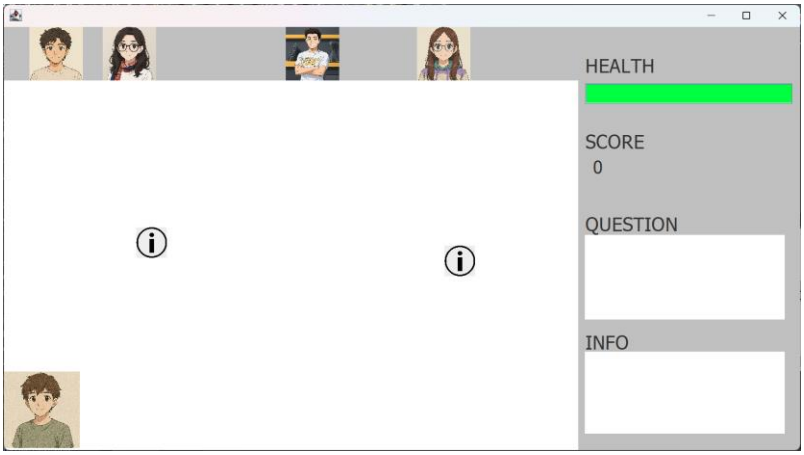
Password does not match regex

Any user, without a need of log in, can see the score board of other users by clicking ‘Open’ button. For this example, I have added some users and game sessions that they have played to demonstrate that score board. For each line, it shows place on score board, username of user that played the game, and his/her score.



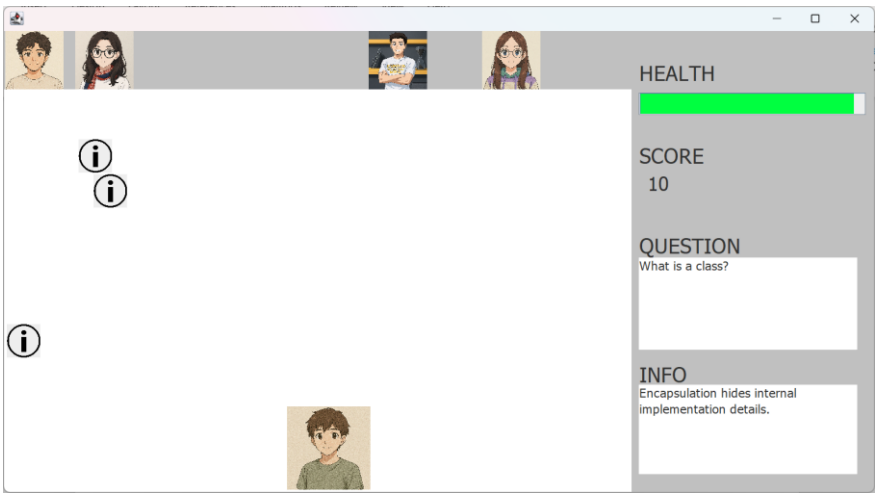
Scoreboard of all users

After logging in, the game immediately starts at level one.



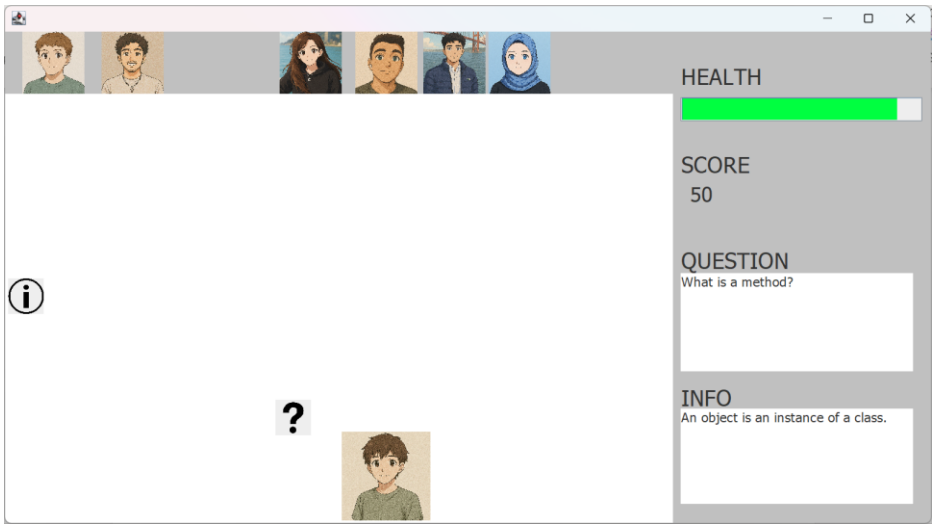
Start of the game: level 1

First, four random SLs are initialized in the top of the screen. The move randomly, 50% percent to right and left. Player is at the bottom of the screen and can be controlled by clicking right arrow or left arrow to move right or left respectively. At the right side of the screen, there is health bar and score field of the player. Also, there is question and info field, where question or info displayed after collecting either question shotbox or information question. *Note: question and info are displayed only when collecting shotbox with that data, both the question and info are overridden when new question or info is collected.* After collecting exactly one question shotbox and one info shotbox we get such image:



Collected one question and info at level 1

From the image, we can see that heath bar decreased by 5%, because we collected question shotbox, and score is increased by 10, because we collected info shotbox. Also, there is a question and info in fields, that we picked randomly from question and info files of the program. After collecting enough points (50 points) to advance to the next level, all the shotboxes from past game are cleared and SLs are removed to add new Knowledge Keepers on the screen opens:



Level 2 with updated enemies (shotboxes on the screen were shot by current Knowledge Keepers)

In this round, there is four SLs that were not used in first round, and two new Teaching Assistants, that move faster than SLs, has low chance to move in the direction of player, and their shotboxes moves faster and deal more damage (-10 health points) and grant more points (+20 points). *Important notice: the Knowledge keepers don't overlap, but they may 'touch' each other.* Log file of the game I have played:

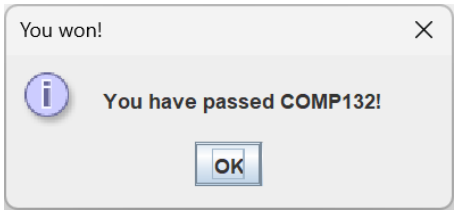
log_game.txt

File Edit View

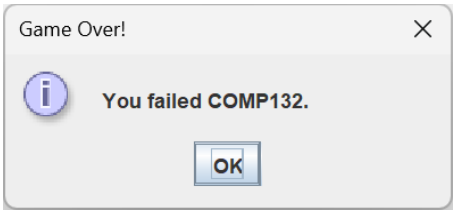
```
[2025-05-27 12:51] Game started by: elnur
[2025-05-27 12:51] User 'elnur' collected info: 'Encapsulation hides internal implementation details.' (+10 points)
[2025-05-27 12:51] Score changed: score: 10; health 100
[2025-05-27 12:52] User 'elnur' hit by question: 'What is a class?' (-5 health)
[2025-05-27 12:52] Health changed: score: 10; health 95
[2025-05-27 13:04] User 'elnur' hit by question: 'What is a method?' (-5 health)
[2025-05-27 13:04] Health changed: score: 10; health 90
[2025-05-27 13:04] User 'elnur' collected info: 'A class defines the blueprint for objects.' (+10 points)
[2025-05-27 13:04] Score changed: score: 20; health 90
[2025-05-27 13:04] User 'elnur' collected info: 'A constructor initializes new objects.' (+10 points)
[2025-05-27 13:04] Score changed: score: 30; health 90
[2025-05-27 13:04] User 'elnur' collected info: 'Encapsulation hides internal implementation details.' (+10 points)
[2025-05-27 13:04] Score changed: score: 40; health 90
[2025-05-27 13:04] User 'elnur' collected info: 'An object is an instance of a class.' (+10 points)
[2025-05-27 13:04] Score changed: score: 50; health 90
```

Log file

After getting enough points (150) to win or completely using health to lose, following window opens:



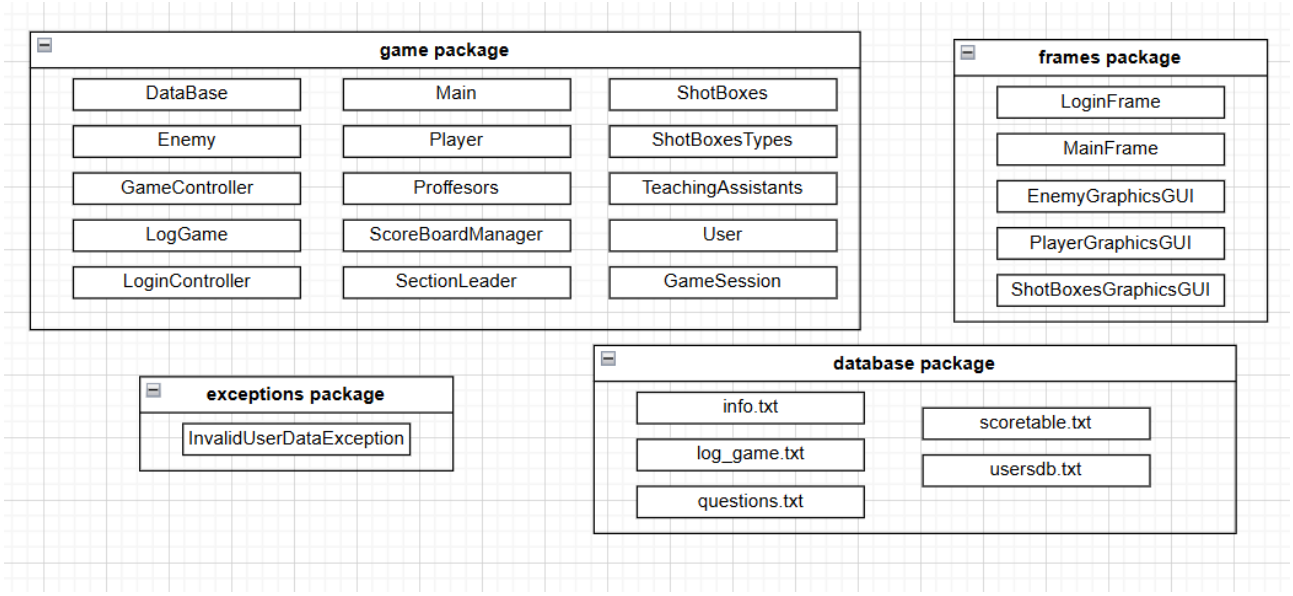
Message after winning the game.



Message after losing the game.

Technical Part | Project Design Description

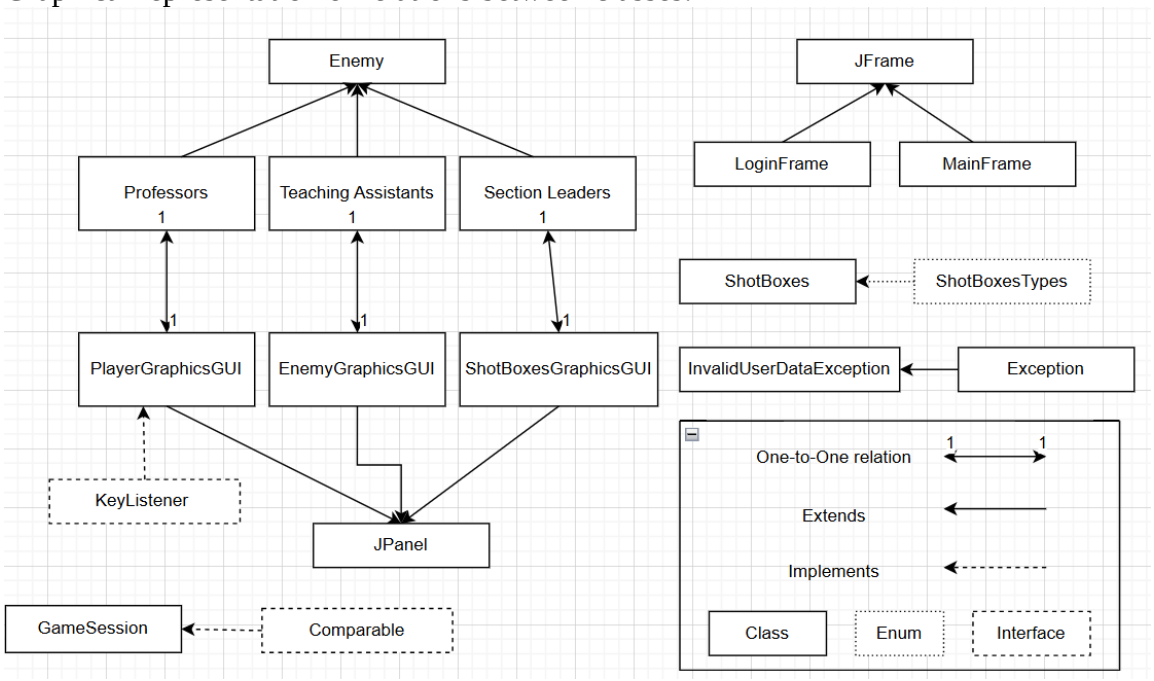
src package's inner packages and files:



Packages of the program

Packages. As visible from the image, src folder has 5 packages: *game, frames, exceptions, database, photos*. *Frames* package is responsible for frames and GUIs of player, enemies and shotboxes. *Database* package contains txt files that keep applications data like score table, information and questions for shotboxes, log file of the game, and users database that keeps track of registered users. *Exceptions* package has custom exceptions. *Game package* has the main logic of the game. It has Controllers, classes to represent player, knowledge keepers, users, mangers, game sessions. *Photos* contains png files of knowledge keepers and players that are used in the program.

Graphical representation of relations between classes:



Relations between classes

Relations. Classes *Professors, TeachingAssistants, SectionLeaders* is an abstract class *Enemy* (extends). Also, *PlayerGraphicsGUI, EnemyGraphicsGUI, ShotBoxesGraphicsGUI* is a *JPanel* (extends). Each of these classes has one-to-one relation with its GUI class, meaning for each object of *Enemy* there is a unique object of *JPanel*. Interface *KeyListener* is implemented by *PlayerGraphicsGUI* so that player can control it through keys. *LoginFrame* and *MainFrame* extends from *JFrame* class. *InvalidUserDataException* extends *Exception* class. An enumeration type class for shotboxes is used in program to specify the type of shotboxes, information or question. GameSession implements Comparable interface.

GUI components used.

GUI components that *LoginFrame* has:

1. Two JTextFields that used for username
2. Two JPasswordFields that used for password
3. Three buttons to Login, register or open score board
4. Three togglebuttons for user to be able to select photo
5. Seven JLabel that contain text to clarify the login page

GUI components that *MainFrame* has:

1. JPanel that contain:
    a. Progress bar for health
    b. Label with score of the user
    c. Two non-editable JTextFields to show text and information

    d. Four label with text to clarify
2. JPanel *EnemyPane* that contains only EnemyGraphicsGUI that initiated only after logging in to the application
3. JPanel *ShotBoxPane* that contains only *ShotBoxesGraphicsGUI* and *PlayerGraphicsGUI* that are initiated only after logging in to the application.

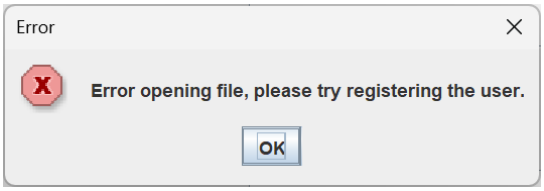
Important implementation idea – in order to generate an enemy or a shotbox to show on the frame, it suffices to create an instance of *Enemy* class or *ShotBoxes* class. By doing that, program not only initializes an object to be referred to, but also a unique *GraphicsGUI* object that is displayed on the frame. Another important notice would be that, as you can see from my class implementation, I have separated GUI part and login of the game into another classes and wrote special functions so that I can clean code the logic of the game, without bothering to much with messy GUI.

FileIO. There are four classes that interact with txt files: *LogGame, ScoreBoardManager and ShotBoxes, LoginController. LogGame* contains method *logEvent(String message)* that can be called from anywhere in the program to log events in *log_game.txt* file. *ScoreBoardManager* has two methods that work with *scoretable.txt* file: *loadScores() and saveScores()*. The former loads all the game sessions from the file, adds it to ArrayList as objects of *GameSession* class. The latter saves all the game sessions in the list to the file in form "name,score". The manager is used in the end of the game – either when player loses by reaching health 0 or winning the game reaching 150 or more score. No game session is wrote to the file when the program is terminated mid-game. In *ShotBoxes* class there is method *readDataForInfoOrQuestion()* that interacts with information.txt and question.txt. Each txt file contains exactly 10 question or info for each type of knowledge keeper and denoted as [SL], [TA], [PR]. The method is called in constructor of shotboxes. Depending on the type of shotbox and the knowledge keeper that shot it, random question or info is selected from a file to be the content of the shotbox. In *LoginController,* two methods interact with *usersdb.txt*: *registerUser(String username, String password, String selectedFile)* and *findByUsername(String username)*. The former method adds the user with data passed as parameter if the provided username is not in database. In order to check whether there exists a user with provided username, the latter method is used. Returns null if username is not found, otherwise returns the username, password and photo url.

Some information on **important classes in project**:

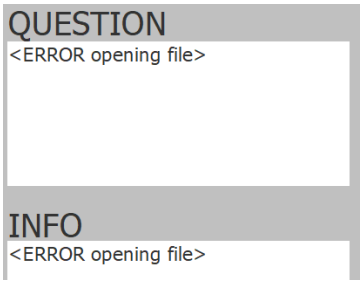
1. GameController class – handles the level initialization, starts timer that moves enemies, shotboxes and checks for collisions.
2. DataBase class – holds lists of SLs, TAs, Professors data and have ready function to pick randomly knowledge keepers for all of three levels.
3. GameSession class – used to record data taken from scoretable.txt as object of this class. Implements Comparable interface so that sorting of game session is viable.
4. LoginController class – handles registration and login of users.
5. ScoreBoardManager class – manager loads past game sessions, sorts them and adds new one.
6. User class – is used for registering and logging in to the program
7. Player class – is used as player in the game to play and interact with.

Error catching. Most of the error catching was done for file input output. The main problem could be that the given file does not exist, got deleted. For example if we delete *usersdb.txt* in program and try to login, it will catch the error and ask user to register, since by registering it will create to file with specified name and only after that the user will be able to log in.



Trying to login when the file is deleted

Similar problem may arise with *question.txt* or *info.txt*. In that case it will print the error to the console log and the question or info field will state the error.



question.txt and info.txt deleted

Additionally, there is custom exception *InvalidUserDataException* that is thrown whenever username or password does not match regex, or when username or password is wrong during log in.

References

1. Java Swing – JPanel With Examples | GeeksforGeeks
2. Java - Timer | Baeldung
3. Using actionPerformed from Another Java Class | GeeksforGeeks
4. How to Write an Action Listener (The Java™ Tutorials > Creating a GUI With Swing > Writing Event Listeners)
5. Java Date and Time